

Question 10: Supermarket Checkout and Stock Consistency

Name: _____ Reg. No: _____

Scenario recap: A supermarket has several checkout counters, each represented as an independent coroutine. Customers may purchase the same product, so the design below guarantees that total stock can never become negative even though several checkouts are processed in an interleaved (quasi-concurrent) fashion.

Design overview: A single shared stock table is owned and updated only by the controlling scheduler - never by a checkout coroutine directly. Each checkout coroutine "scans" one product at a time and uses `coroutine.yield()` to hand the request over to the scheduler, then blocks until it is resumed with the scheduler's decision. Because Lua coroutines are cooperative, only one piece of code is ever running at any instant, so the check-then-update logic in `checkStock()` can never be split across two checkouts - this is what keeps stock logically consistent.

(a) Checkout as a coroutine that yields after every scan - 4 marks

Each checkout is created with `coroutine.create()` wrapping a function that loops over its shopping list. After "scanning" each item it calls `coroutine.yield(item.product, item.qty)`, pausing execution and handing the request to the scheduler. The coroutine's local loop variables (its position in the cart) are preserved automatically across yields, so it resumes exactly where it left off.

(b) Shared stock table in the controlling scheduler - 4 marks

The stock table is declared once, in the scheduler's scope, and is never passed into or mutated inside a checkout coroutine. This gives the whole program a single authoritative record of what is available, avoiding the duplicated or stale copies that could otherwise cause inconsistent totals.

(c) Deciding whether a purchase can proceed before updating stock - 4 marks

The function `checkStock(product, qty)` is the only place stock is ever changed. It first checks whether `stock[product] >= qty`; only if that holds does it subtract the quantity and report success. If not, it rejects the request and returns the quantity actually available, leaving stock untouched. Because this check-and-update happens inside one uninterrupted function call, no other checkout can slip in between the check and the update.

(d) Interleaved execution with consistent stock - 4 marks

The scheduler keeps a list of active checkout coroutines and resumes them in round-robin order: one turn per checkout per pass, until every coroutine reaches the dead state. This means Checkout-1, Checkout-2 and Checkout-3 all make progress in an interleaved order rather than one finishing completely before the next starts - directly demonstrating quasi-concurrent execution. In the sample run below, all three checkouts contend for milk in the same pass; the third request is correctly rejected once stock is insufficient, and the closing stock count for every product remains non-negative.

(e) Why single-threaded coroutines simplify stock control - 4 marks

In a truly multithreaded system, the operating system can pre-empt a thread at any machine instruction, so two threads could both read the same stock value before either writes back a new one - a classic race condition that normally requires locks, mutexes or atomic operations to prevent. With coroutines, a

context switch only ever happens where the programmer explicitly writes `coroutine.yield()`. Since that call is never placed inside `checkStock()`, the check-then-update sequence is effectively atomic by construction, and correctness is guaranteed without any synchronisation primitives.

Full Lua Solution

```
-- =====
-- CCS 2105 - Question 10
-- Supermarket Checkout and Stock Consistency
-- =====
-- Design idea:
-- * Each checkout counter is its own coroutine. It never touches
--   the stock table directly - it only *asks* for permission.
-- * The shared stock table lives in the scheduler's scope, so
--   there is exactly one authoritative copy of the truth.
-- * The scheduler resumes checkouts in round-robin order. Because
--   only one coroutine can ever be running at a time, the
--   check-then-update sequence in checkStock() can never be
--   interrupted midway by another checkout - this is what keeps
--   stock logically consistent without locks.
-- =====

-- (b) Shared stock table - single source of truth, owned by the scheduler
local stock = {
    milk      = 12,
    bread     = 8,
    rice      = 15,
    sugar     = 6,
    cooking_oil = 4
}

-- Each checkout's shopping list: a sequence of {product, qty} requests
local checkoutData = {
    {
        name = "Checkout-1 (Amina)",
        cart = {
            { product = "milk",   qty = 3 },
            { product = "bread",  qty = 2 },
            { product = "sugar",  qty = 5 }, -- will be contested / may fail
        }
    },
    {
        name = "Checkout-2 (Otieno)",
        cart = {
            { product = "milk",    qty = 4 },
            { product = "rice",    qty = 6 },
            { product = "cooking_oil", qty = 2 },
        }
    },
    {
        name = "Checkout-3 (Wanjiru)",
        cart = {
            { product = "milk",   qty = 6 }, -- contests milk with 1 & 2
            { product = "sugar",  qty = 2 },
            { product = "bread",  qty = 3 },
        }
    }
}

-- (a) Each checkout is a coroutine that scans one product at a time and
--   yields the request out to the scheduler; it waits for the
--   scheduler's verdict before printing a receipt line and moving on.
local function checkoutProcess(name, cart)
    return coroutine.create(function()
        for _, item in ipairs(cart) do
            local approved, remaining = coroutine.yield(item.product, item.qty)

            if approved then
                print(string.format(
                    "[%s] Scanned %d x %-11s -> APPROVED (stock left: %d)",
                    name, item.qty, item.product, remaining))
            else
                print(string.format(
                    "[%s] Scanned %d x %-11s -> REJECTED (only %d in stock)",
                    name, item.qty, item.product, remaining))
            end
        end
    end)
end
```

```

        end
        return "receipt closed for " .. name
    end)
end

-- (c) checkStock() is the single gatekeeper. It decides BEFORE any
--     update whether a purchase can proceed, then performs the update
--     as one atomic step (no coroutine can pre-empt this function).
local function checkStock(product, qty)
    local available = stock[product] or 0
    if available >= qty then
        stock[product] = available - qty
        return true, stock[product]
    else
        return false, available
    end
end

-- Build the coroutine list
local checkouts = {}
for _, c in ipairs(checkoutData) do
    table.insert(checkouts, { name = c.name, co = checkoutProcess(c.name, c.cart) })
end

print("=== Opening stock ===")
for _, p in ipairs({ "milk", "bread", "rice", "sugar", "cooking_oil" }) do
    print(string.format(" %-11s : %d", p, stock[p]))
end
print()
print("=== Interleaved checkout processing ===")

-- (d) Round-robin scheduler demonstrating interleaved execution.
--     Each pass gives every live checkout exactly one turn, so all
--     three counters progress "quasi-simultaneously" while stock
--     stays consistent, since real updates only ever happen one at a
--     time inside checkStock().
local pendingArgs = {} -- results to feed back into each coroutine on its next resume
local active = #checkouts

while active > 0 do
    for _, entry in ipairs(checkouts) do
        if coroutine.status(entry.co) ~= "dead" then
            local args = pendingArgs[entry.co] or {}
            local ok, a, b = coroutine.resume(entry.co, table.unpack(args))

            if not ok then
                -- Error handling: never let one bad checkout crash the whole system
                print(string.format("[%s] ERROR: %s", entry.name, tostring(a)))
                active = active - 1
            elseif coroutine.status(entry.co) == "dead" then
                print(string.format("[%s] %s", entry.name, tostring(a)))
                active = active - 1
            else
                local product, qty = a, b
                local approved, remaining = checkStock(product, qty)
                pendingArgs[entry.co] = { approved, remaining }
            end
        end
    end
end

print()
print("=== Closing stock ===")
local negative_found = false
for _, p in ipairs({ "milk", "bread", "rice", "sugar", "cooking_oil" }) do
    print(string.format(" %-11s : %d", p, stock[p]))
    if stock[p] < 0 then negative_found = true end
end

print()
if negative_found then
    print("CONSISTENCY CHECK: FAILED - stock went negative!")
else
    print("CONSISTENCY CHECK: PASSED - no product ever went negative.")
end
end

```

Sample Terminal Output (produced by running the script above)

```
=== Opening stock ===
milk      : 12
bread     : 8
rice      : 15
sugar     : 6
cooking_oil : 4
```

```
=== Interleaved checkout processing ===
[Checkout-1 (Amina)] Scanned 3 x milk      -> APPROVED (stock left: 9)
[Checkout-2 (Otieno)] Scanned 4 x milk     -> APPROVED (stock left: 5)
[Checkout-3 (Wanjiru)] Scanned 6 x milk    -> REJECTED (only 5 in stock)
[Checkout-1 (Amina)] Scanned 2 x bread     -> APPROVED (stock left: 6)
[Checkout-2 (Otieno)] Scanned 6 x rice     -> APPROVED (stock left: 9)
[Checkout-3 (Wanjiru)] Scanned 2 x sugar   -> APPROVED (stock left: 4)
[Checkout-1 (Amina)] Scanned 5 x sugar     -> REJECTED (only 4 in stock)
[Checkout-1 (Amina)] receipt closed for Checkout-1 (Amina)
[Checkout-2 (Otieno)] Scanned 2 x cooking_oil -> APPROVED (stock left: 2)
[Checkout-2 (Otieno)] receipt closed for Checkout-2 (Otieno)
[Checkout-3 (Wanjiru)] Scanned 3 x bread   -> APPROVED (stock left: 3)
[Checkout-3 (Wanjiru)] receipt closed for Checkout-3 (Wanjiru)
```

```
=== Closing stock ===
milk      : 5
bread     : 3
rice      : 9
sugar     : 4
cooking_oil : 2
```

CONSISTENCY CHECK: PASSED - no product ever went negative.

Note: Run `lua5.4 q10_checkout.lua` (or `lua q10_checkout.lua` depending on your install) from a terminal in the folder containing the script to reproduce this output yourself.